دانشگاه اصفهان
دانشکده مهندسی کامپیوتر

پروژه‌ی اول



UNO

درس

ساختمان داده‌ها

بخش ۳ | طراحی فنی و پیاده‌سازی پروژه

نام استاد
دکتر افسانه فاطمی

نیمسال تحصیلی
پاییز ۴۰۵

اعضای تیم



عرفان رحمت



مریم زارع‌پور



عارف زرگر



مهسا مظفری

فهرست مطالب



۳ طراحی فنی و پیاده‌سازی پروژه ۳

۳.۱ معماری لایه‌ای: منطق بازی جدا از نمایش ۳

۳.۲ جدول نگاشت ساختمان داده‌های الزامی ۴

۳.۳ طراحی کلاس‌های موجودیت و مدیریتی ۷

۳.۴ منطق موتور بازی، بازسازی Deck و چرخه نوبت ۸

۳.۵ الزامات پیاده‌سازی بازگشتی ۹

۳.۶ منطق تصمیم‌گیری خودکار بازیکن ربات ۹

۳.۷ رابط کاربری کنسول، منوها و ساختار دستورات ۱۰

۳.۸ دستگاه پرتاب کارت Draw ۱۲

۳.۹ اصول کدنویسی تمیز و نکات تحویل ۱۲

۳.۱۰ قابلیت‌های امتیازی ۱۳

بخش ۳

طراحی فنی و پیاده‌سازی پروژه



در بخش ۲، قوانین و منطق بازی **UNO** به‌طور کامل تشریح شد. آنچه در این بخش دنبال می‌شود، تبدیل این قوانین به یک سیستم نرم‌افزاری ماژولار، ساختارمند و قابل‌اجرا است. محور اصلی این بخش، **پیاده‌سازی دستی ساختمان داده‌های پایه** و به‌کارگیری معنادار هر یک در جایگاه مناسب خود است: مدیریت کارت‌ها، گردش نوبت، جریمه‌ها، محاسبه‌ی امتیازات و رتبه‌بندی کاربران.

قانون طلایی: ممنوعیت مطلق استفاده از ساختمان داده‌های آماده

تمامی **ساختمان داده‌های اصلی** این پروژه (`SinglyLinkedList`، `DoublyCircularLinkedList`، `CustomStack`، `CustomQueue` و ...) و همچنین **الگوریتم‌ها** باید به‌صورت دستی و از صفر پیاده‌سازی شوند؛ تنها با استفاده از `Node` و آرایه‌های خام پایه. استفاده از کالکشن‌ها و کتابخانه‌های آماده‌ی زبان در این بخش‌ها **اکیداً ممنوع** است و نمره‌ی بخش مربوطه صفر لحاظ خواهد شد؛ از جمله `ArrayList` و `Arrays.sort` در جاوا، `STL` در C++، یا متدهای آماده در پایتون.

زبان برنامه‌نویسی و رابط کاربری (GUI / CLI)

انتخاب زبان برنامه‌نویسی از میان زبان‌های شیءگرا (مانند Java، C++، C# یا Python) آزاد است. پیاده‌سازی نسخه‌ی پایه در محیط خط فرمان (CLI) الزامی است و همچنین طراحی رابط کاربری گرافیکی (GUI) به‌عنوان بخش امتیازی در نظر گرفته می‌شود.

۳.۱ معماری لایه‌ای: منطق بازی جدا از نمایش (MVC & Clean Architecture)

برای جلوگیری از کدنویسی اسپاگتی و انباشته شدن تمام منطق در یک فایل، برنامه باید در سه لایه‌ی مستقل طراحی شود (الگوی MVC) تا هسته‌ی بازی هیچ وابستگی مستقیمی به نحوه‌ی چاپ در کنسول یا گرافیک نداشته باشد.

۱. لایه‌ی Model

داده‌های بازی را در خود نگه می‌دارد؛ ساختمان داده‌های دست‌ساز و موجودیت‌های پایه (کارت، بازیکن، حساب کاربری و راند). این لایه صرفاً ساختار داده‌ها را تعریف می‌کند و منطق تصمیم‌گیری ندارد.

۲. لایه‌ی Controller

هسته‌ی منطقی بازی است. مدیریت دسته، چرخش نوبت، بررسی قانونی بودن حرکت‌ها و محاسبه‌ی امتیاز بر عهده‌ی این لایه است.

۳. لایه‌ی View

تنها لایه‌ای است که با کاربر تعامل مستقیم دارد: نمایش منوها و وضعیت میز، و دریافت دستورات. این لایه هرگز نباید قوانین بازی را بررسی کند یا تغییری در دست بازیکنان ایجاد نماید.

View | نمایش و ورودی ← Controller | منطق بازی ← Model | داده‌ها

فلش‌ها نشان‌دهنده‌ی جهت وابستگی بین لایه‌ها هستند و این جهت هرگز معکوس نمی‌شود. عملیات چاپ و دریافت ورودی تنها در لایه‌ی **View** مجاز است. به‌عنوان معیاری ساده برای بررسی صحت معماری، کافی است در نظر بگیرید که در صورت جایگزینی کنسول با رابط گرافیکی، لایه‌های **Controller** و **Model** نباید تغییر کنند.

۳.۲ جدول نگاشت ساختمان داده‌های الزامی

در جدول زیر، محل دقیق استفاده از هر ساختمان داده در پروژه و پیچیدگی زمانی مورد انتظار برای عملیات‌های اصلی آن مشخص شده است:

بخش بازی	ساختمان داده الزامی	کاربرد دقیق در پروژه	مرتبه زمانی مجاز
دست بازیکن	`SinglyLinkedList`	نگهداری کارت‌های دست هر بازیکن؛ درج کارت کشیده‌شده در انتها و حذف کارت بازی‌شده با اندیس	`Add: O(1)` `Remove: O(k)`
گردش نوبت بازیکنان	`DoublyCircularLinkedList`	نگهداری حلقه بازیکنان؛ انتقال به نفر بعدی/قبلی، پرش نوبت (Skip) و معکوس کردن جهت (Reverse)	`Reverse: O(1)` `Next: O(1)`
دسته کشیدنی (Deck)	`CustomQueue`	نگهداری کارت‌های برزده‌شده و تحویل کارت از ابتدای صف به بازیکنان هنگام کشیدن کارت	`Enqueue: O(1)` `Dequeue: O(1)`
کارت‌های بازی‌شده (DiscardPile)	`CustomStack`	نگهداری کارت‌های روی میز و مشاهده فوری کارت فعلی روی میز	`Push: O(1)` `Peek/Pop: O(1)`
زنجیره جریمه (Stacking)	`CustomStack`	نگهداری مقادیر جریمه (+4 / +2) در جریان تجمع (PenaltyStack) و تخلیه بازگشتی در پایان زنجیره	`Push: O(1)` `Sum: O(m)`
حساب‌ها و Leaderboard	`DynamicArray`	نگهداری حساب‌های کاربری و مرتب‌سازی جدول رده‌بندی با الگوریتم بازگشتی	`Access: O(1)` `Sort: O(U log U)`
تاریخچه امتیازات راند	`SinglyLinkedList`	ثبت امتیاز کسب‌شده توسط بازیکن در هر راند به صورت مجزا از مجموع امتیاز کل بازی	`Add: O(1)`

تحلیل انتخاب ساختمان داده‌ها در جدول بالا

دو ساختار در این جدول بیش از یک بار به کار رفته‌اند: `SinglyLinkedList` در دست بازیکن و تاریخچه‌ی امتیازات، و `CustomStack` در `DiscardPile` و `PenaltyStack`. این تکرار تصادفی نیست؛ معیار انتخاب، الگوی عملیات است نه کاربرد نهایی. پیاده‌سازی این ساختارها در کل پروژه یکسان است و تفاوت تنها در منطق مدیریتی لایه‌ی Controller ظاهر می‌شود. جزئیات پیاده‌سازی هر ساختمان داده در بخش‌های ۳.۲.۱ تا ۳.۲.۵ بررسی می‌شود.

۳.۲.۱ لیست پیوندی یک‌طرفه (SinglyLinkedList<T>) — دست بازیکن و امتیازات

تعداد کارت‌های دست هر بازیکن در طول بازی مدام تغییر می‌کند و بازیکن می‌تواند هر کارتی را از ابتدا، وسط یا انتهای دست خود انتخاب و بازی کند. برای تضمین درج در زمان O(1)، کلاس لیست پیوندی باید علاوه بر اشاره‌گر `head`، اشاره‌گر `tail` و متغیر `size` را نیز نگهداری کند. همچنین برای نمایش مرتب کارت‌های دست بازیکن، این کلاس باید یک متد مرتب‌سازی بازگشتی نیز در اختیار داشته باشد.



- **فیلدهای کلاس** `Node<T>`: شامل فیلد داده `T data` و اشاره‌گر `Node<T> next`.
- **متدهای الزامی کلاس** `SinglyLinkedList<T>`:
  - `void addLast(T data)`: افزودن عنصر جدید به انتهای لیست با استفاده از اشاره‌گر `tail` در زمان O(1).
  - `T removeAt(int index)`: حذف گره موجود در اندیس `index`، تنظیم مجدد اشاره‌گرهای `head` / `tail` در صورت حذف گره اول یا آخر، و بازگرداندن داده حذف‌شده در زمان O(k).
  - `T get(int index)`: بازگرداندن عنصر موجود در اندیس مشخص بدون حذف آن در زمان O(k).
  - `void sort(Comparator<T> cmp)`: مرتب‌سازی عناصر لیست بر اساس معیار مقایسه‌ی داده‌شده با الگوریتم بازگشتی Merge Sort در زمان O(k log k). این متد در دست بازیکن برای مرتب‌سازی کارت‌ها (ابتدا بر اساس رنگ، سپس نوع و مقدار) و در تاریخچه‌ی امتیازات راند برای مرتب‌سازی بر اساس مقدار امتیاز به کار می‌رود.
  - `Node<T> getHead()`، `int size()` و `boolean isEmpty()`: دسترسی به سرلیست (برای پیمایش بازگشتی امتیازات) و بررسی خالی شدن دست بازیکن در زمان O(1).

۳.۲.۲ لیست پیوندی دوطرفه حلقوی (DoublyCircularLinkedList) — گردش نوبت

در بازی UNO، بازیکنان دور یک میز حلقوی می‌نشینند؛ نوبت پس از آخرین نفر به نفر اول برمی‌گردد و کارت `Reverse` جهت چرخش نوبت را برعکس می‌کند. در این لیست، هر گره (`DoublyNode`) شامل شیء `Player` و دو اشاره‌گر `next` (ساعت‌گرد) و `prev` (پادساعت‌گرد) است و در لیست ناتهی هیچ اشاره‌گری `null` نیست.



تغییر جهت در زمان O(1) و قانون خاص بازی ۲ نفره

کلاس `TurnManager` یک اشاره‌گر `current` و یک متغیر بولین `isClockwise` نگه می‌دارد. با بازی شدن `Reverse` در بازی‌های ۳ نفر به بالا، فقط مقدار `isClockwise = !isClockwise` تغییر می‌کند (O(1)) و نوبت ۱ گام در جهت جدید جلو می‌رود. **اما در بازی ۲ نفره** (`size == 2`)، طبق قوانین بخش ۲، کارت `Reverse` دقیقاً مانند `Skip` عمل کرده و با ۲ گام جلو رفتن، نوبت مجدداً به همان بازیکن می‌رسد.

- **متدهای الزامی در** `TurnManager`: `addPlayer(Player p)`، `getCurrentPlayer()`، `peekNextPlayer()` (مشاهده نفر بعدی بدون تغییر نوبت)، `reverseDirection()` و `advanceTurn(int steps)` (حرکت به تعداد `steps` گام بر روی اشاره‌گر `next` یا `prev` بسته به جهت فعلی).

۳.۲.۳ صف دست‌ساز (CustomQueue<T>) — مدیریت دسته کشیدنی (Deck)

کارت‌های کشیدنی پس از بر زدن (Shuffle) وارد یک صف می‌شوند تا اولین کارت قرارگرفته در بالای دسته، اولین کارتی باشد که توسط بازیکن کشیده می‌شود. صف باید با لیست پیوندی (دارای دو اشاره‌گر `front` و `rear`) یا آرایه حلقوی پیاده‌سازی شود تا هر دو عملیات درج و حذف در زمان O(1) انجام شوند.

- **متدهای الزامی:** `void enqueue(T item)` (افزودن به انتهای صف در rear)، `T dequeue()` (حذف و بازگرداندن عنصر ابتدای صف از `front`)، `T peek()`، `boolean isEmpty()` و `int size()`.

۳.۲.۴ پشته دست‌ساز (CustomStack<T>) — دورریز (DiscardPile) و جریمه‌ها (PenaltyStack)

ساختمان داده پشته با نگهداری اشاره‌گر `top`، دسترسی و درج در بالای پشته را در زمان O(1) فراهم می‌کند و در دو بخش مجزا به کار می‌رود:

۱. پشته کارت‌های بازی‌شده (DiscardPile)

نگهداری کارت‌هایی که بازیکنان وسط میز می‌اندازند. آخرین کارت بازی‌شده همواره با `peek()` در بالای پشته قرار دارد و مبنای بررسی حرکت بعدی است.

۲. پشته تجمع جریمه‌ها (PenaltyStack)

هنگام بازی شدن کارت `+2` یا `+4`، عدد جریمه (`2` یا `4`) در این پشته `push` می‌شود تا در پایان زنجیره Stacking به صورت بازگشتی تخلیه و جمع زده شود.

- **متدهای الزامی:** `void push(T item)`، `T pop()`، `T peek()`، `boolean isEmpty()` و `int size()` (همگی از مرتبه O(1)).

۳.۲.۵ آرایه پویا (DynamicArray<T>) — مدیریت حساب‌ها و Leaderboard

برای نگهداری لیست کاربران ثبت‌نام‌شده و اجرای الگوریتم مرتب‌سازی جدول رده‌بندی، باید یک کلاس آرایه پویا بر پایه آرایه خام اولیه (با ظرفیت پیش‌فرض مثلاً ۱۰) پیاده‌سازی کنید.

- **مکانیزم تغییر اندازه خودکار (Resize):** هرگاه تعداد عناصر (`size`) به ظرفیت آرایه داخلی (`capacity`) برسد، متد `resize()` آرایه‌ای جدید با ظرفیت **۲ برابر** (`2 * capacity`) تخصیص داده و عناصر قبلی را به آن کپی می‌کند (O(1)).
- **متدهای الزامی:** `void add(T item)`، `T get(int index)`، `void set(int index, T item)`، `void swap(int i, int j)` و `int size()`.

۳.۳ طراحی کلاس‌های موجودیت و مدیریتی

برای رعایت اصل تک‌مسئولیتی (SRP)، کلاس‌های برنامه به دو گروه **موجودیت‌های داده** (Entities) و **مدیران منطق** (Managers) تقسیم می‌شوند:

الف) کلاس‌های موجودیت پایه (Models / Entities)

نام کلاس	فیلدها و ویژگی‌های اصلی	مسئولیت و قرارداد طراحی شیءگرا

`Card`	`Color color`
	`CardType type`
	`int value` (۰-۹، ۲۰ یا ۵۰)
		نگهداری مشخصات یک کارت. برای کارت‌های Wild، رنگ اولیه `NONE` است. **نکته مهم:** کلاس Card صرفاً نگه‌دارنده داده است و نباید متد اجرای قوانین درون آن باشد.

`Account`	`username`, `password`, `displayName`, `totalScore`, `gamesPlayed`, `wins`	نگهداری اطلاعات پایدار حساب کاربری و آمار کل بازی‌های انجام‌شده برای نمایش و مرتب‌سازی در `Leaderboard`.

(Abstract) `Player`
`HumanPlayer`
`BotPlayer`
	`Account account`, `String name`
	`SinglyLinkedList<Card> hand`
	`SinglyLinkedList<Integer> roundScores`
	`int currentGameScore`
		کلاس پایه بازیکن شامل عملیات دریافت/حذف کارت از دست است. زیرکلاس‌های انسان و ربات متد انتزاعی `chooseAction(RoundState)` را به شکل چندریخت (Polymorphic) پیاده‌سازی می‌کنند.

ب) کلاس‌های مدیریتی و موتور بازی (Engine & Managers)

کلاس مدیریتی	ساختمان داده / وابستگی اصلی	وظایف و متدهای کلیدی در موتور بازی
`Deck` & `DiscardPile`	`CustomQueue<Card> drawQueue` `CustomStack<Card> playedStack`	ساخت ۱۰۸ کارت استاندارد، بر زدن (`shuffle`)، توزیع ۷ کارت به هر نفر، تحویل کارت (`drawCard`) و بازسازی خودکار `Deck` از روی `DiscardPile` در صورت خالی شدن.
`TurnManager`	`DoublyCircularLinkedList<Player>` `boolean isClockwise`	مدیریت بازیکن فعلی، تغییر جهت گردش با کارت `Reverse`، پرش نوبت در کارت `Skip` و مدیریت قانون خاص بازی‌های ۲ نفره.
`RuleManager`	بدون حالت پایدار (Stateless Validator)	متد `isValidPlay(card, topCard, activeColor, hand, penaltyStack)` برای بررسی اعتبار حرکت، بررسی شرط مجاز بودن `WildDrawFour` و اعمال اثر کارت‌ها.
`ScoreManager` & `Leaderboard`	`DynamicArray<Account> accounts`	محاسبه بازگشتی امتیاز دست بازندگان در پایان راند، ثبت در `roundScores`، بررسی شرط برنده نهایی بازی و مرتب‌سازی بازگشتی جدول رده‌بندی کاربران.
`Game` & `Round`	`activeColor`, `PenaltyStack`, `targetLimit`	کلاس `Round` یک دست واحد را تا خالی شدن دست یک بازیکن اجرا می‌کند؛ کلاس `Game` دست‌های متوالی را تا تحقق شرط پایان کل بازی (امتیاز یا تعداد دست) مدیریت می‌کند.

۳.۴ منطق موتور بازی، بازسازی Deck و چرخه نوبت

۱. ساخت دسته، بر زدن (Shuffle) و بازسازی خودکار Deck

در ابتدای هر راند، ۱۰۸ کارت استاندارد (طبق جدول بخش ۲.۳) درون یک آرایه موقت ساخته شده، با الگوریتم **Fisher-Yates Shuffle** به صورت تصادفی جابه‌جا می‌شوند و وارد صف `Deck` می‌گردند. سپس به هر بازیکن ۷ کارت داده شده و کارت اول روی `DiscardPile` قرار می‌گیرد (اگر کارت اول `WildDrawFour` باشد، به دسته برگشته، بر زده می‌شود و کارت دیگری کشیده می‌شود). در طول بازی، هرگاه نیاز به کشیدن کارت باشد و `Deck.isEmpty()` برقرار شود، مراحل زیر فوراً اجرا می‌گردد:

- **گام ۱ (نگهداری کارت فعلی میز):** کارت بالای پشته `DiscardPile` با `pop()` خارج شده و در متغیر موقت `topCard` حفظ می‌شود.
- **گام ۲ (تخلیه، ریست رنگ و بر زدن):** تمامی کارت‌های زیرین `DiscardPile` خارج شده، رنگ کارت‌های `Wild` به `NONE` برگردانده می‌شود، کارت‌ها درون آرایه بر زده شده و مجدداً در صف `Deck` درج (`enqueue`) می‌شوند.
- **گام ۳ (بازگردانی کارت فعلی):** کارت `topCard` دوباره روی پشته `DiscardPile` قرار می‌گیرد (`push`) تا کارت و `activeColor` میز تغییر نکند.

۲. چرخه‌ی اجرای یک نوبت

هر نوبت از یک مسیر ثابت می‌گذرد و این مسیر برای بازیکن انسانی و ربات یکسان است؛ تفاوت آن‌ها فقط در نحوه‌ی تصمیم‌گیری است:



پس از معتبر شناخته شدن حرکت، کارت با `removeAt` از دست بازیکن خارج می‌شود، روی `DiscardPile` قرار می‌گیرد و اثر آن طبق جدول زیر اعمال می‌شود. اگر حرکت نامعتبر باشد، نوبت مصرف نمی‌شود و از بازیکن دوباره ورودی گرفته می‌شود:

کارت بازی‌شده	اثر روی وضعیت بازی	حرکت نوبت پس از اثر
Skip	بازیکن بعدی نوبتش را از دست می‌دهد.	۲ گام
Reverse	در بازی ۳ نفره و بیشتر جهت گردش عوض می‌شود؛ در بازی ۲ نفره مانند `Skip` عمل می‌کند.	۱ گام (۲ نفره: ۲ گام)
+2 و +4	عدد ۲ یا ۴ در `PenaltyStack` درج می‌شود و بازیکن بعدی باید زنجیره را ادامه دهد یا مجموع جریمه را بکشد.	۱ گام
Wild	رنگ انتخاب‌شده‌ی بازیکن در `activeColor` ثبت می‌شود.	۱ گام

۳.۵ الزامات پیاده‌سازی بازگشتی

طبق سرفصل‌های درس ساختمان داده‌ها، شما موظف هستید در **۳ بخش مشخص زیر**، الگوریتم‌های خود را به‌صورت **کاملاً بازگشتی** و بدون استفاده از حلقه‌های تکرار (`for` / `while`) پیاده‌سازی نمایید:

۱

تخلیه و جمع پشته جریمه

شرط پایه: اگر `penaltyStack.isEmpty()` برقرار بود، مقدار `0` برگردانده شود.

گام بازگشتی: یک عنصر با `pop()` خارج شده و با حاصل فراخوانی بازگشتی تابع روی پشته جمع شود:

```
return stack.pop() +
sumPenalty(stack)
```

.

۲

محاسبه امتیاز دست بازندگان

شرط پایه: اگر گره ورودی `node == null` بود، مقدار `0` برگردانده شود.

گام بازگشتی: امتیاز کارت فعلی (عددی: عدد کارت، عملیاتی: ۲۰، ۵۰ :Wild) با `calcHandScore(node.next)` جمع شود.

۳

مرتب‌سازی بازگشتی رتبه‌ها

در کلاس `Leaderboard` ، مرتب‌سازی `DynamicArray<Account>` بر اساس مجموع امتیاز کل (و در صورت تساوی، تعداد برد بیشتر) باید با الگوریتم تقسیم و غلبه بازگشتی **Merge Sort** یا **Quick Sort** در زمان **O(U log U)** پیاده شود.

۳.۶ منطق تصمیم‌گیری خودکار بازیکن ربات

کلاس `BotPlayer` از `Player` ارث‌بری می‌کند و در نوبت خود بدون نیاز به ورودی کنسول، طبق الگوریتم قطعی زیر تصمیم می‌گیرد و حرکتش را (که در کنسول نیز لاگ می‌شود) به موتور بازی ارسال می‌کند:

۱

رفتار در زمان وجود جریمه فعال (PenaltyStack ناتهی)

ربات لیست پیوندی دست خود را از اندیس ۰ پیمایش می‌کند؛ اگر کارت `DrawTwo (+2)` یا `WildDrawFour (+4)` داشته باشد، اولین مورد یافت‌شده را بازی می‌کند تا جریمه را به نفر بعدی منتقل کند؛ در غیر این صورت، دستور `draw` را اجرا کرده و جریمه را می‌کشد.

۲

انتخاب کارت در نوبت عادی (اولویت‌بندی حرکت)

ربات کارت‌های معتبر دست خود را بررسی می‌کند. **اولویت اول** با کارت‌های رنگی (عددی و عملیاتی هم‌رنگ با `activeColor` یا هم‌عدد/هم‌نوع با کارت روی میز) است. اگر هیچ کارت رنگی معتبری نداشت، به سراغ کارت `Wild` و در نهایت `WildDrawFour` (با رعایت شرط نداشتن رنگ فعلی) می‌رود.

۳

کشیدن کارت در صورت نبود حرکت معتبر

اگر هیچ کارت قابل بازی در دست نداشت، یک کارت از `Deck` می‌کشد؛ در صورتی که کارتِ تازه‌کشیده‌شده طبق قوانین قابل بازی باشد، ربات بلافاصله آن را در همان نوبت بازی می‌کند.

۴

الگوریتم انتخاب رنگ بهینه برای کارت‌های Wild

هرگاه ربات کارت `Wild` یا `WildDrawFour` بازی کند، دست خود را پیمایش کرده، تعداد کارت‌های هر ۴ رنگ اصلی را می‌شمارد و **رنگی که بیشترین فراوانی را در دست او دارد** به‌عنوان رنگ جدید بازی (`activeColor`) انتخاب می‌کند (در صورت تساوی یا خالی بودن، به ترتیب اولویت `RED, BLUE, GREEN, YELLOW`).

۳.۷ رابط کاربری کنسول (CLI)، منوها و ساختار دستورات

در نسخه پایه، تعامل کاربر با برنامه از طریق خط فرمان (Console) انجام می‌شود. برنامه دارای سه محیط اصلی است: **۱. منوی احراز هویت**، **۲. منوی اصلی کاربری** و **۳. محیط میز بازی**. دستورات هر بخش فقط در همان منو معتبر هستند:

۱. دستورات منوی احراز هویت و منوی اصلی

`register -u <username> -p <password> -n <display_name>`	ساخت حساب کاربری جدید (نام کاربری باید بدون فاصله و یکتا باشد؛ رمز عبور حداقل ۴ کاراکتر)
`login -u <username> -p <password>`	ورود به حساب کاربری و انتقال به منوی اصلی برنامه
`leaderboard`	نمایش جدول رده‌بندی تمامی کاربران ثبت‌شده (رتبه، نام نمایشی، نام کاربری، امتیاز کل، تعداد بازی و برد)
`start_game -p <humans> -b <bots> -mode <score|rounds> -limit <N>`	شروع بازی جدید؛ مثلاً `start_game -p 1 -b 3 -mode score -limit 500` (مجموع بازیکنان: حداقل ۲ و حداکثر ۱۰ نفر)
`logout | exit`	خروج از حساب کاربری فعلی و بازگشت به منوی ورود، یا بستن کامل برنامه

نحوه ورود نام بازیکنان در بازی‌های چندنفره انسانی

اگر در دستور `start_game` تعداد بازیکنان انسانی بیش از ۱ نفر تعیین شود (`humans > 1`)، برنامه پیش از شروع راند اول، نام کاربری بازیکنان انسانی دیگر را به ترتیب دریافت می‌کند تا امتیاز نهایی بازی در حساب کاربری آن‌ها نیز ثبت گردد. ربات‌ها به صورت خودکار با نام‌های `Bot_1` ، `Bot_2` و... ساخته می‌شوند.

۲. نمونه خروجی استاندارد میز بازی در کنسول

در ابتدای هر نوبت بازیکن انسانی، وضعیت کامل میز، جهت گردش، تعداد کارت‌های هر بازیکن، کارت و رنگ فعلی و لیست کارت‌های دست بازیکن به همراه اندیس آن‌ها باید به فرمت خوانای زیر چاپ شود:

```
================================================================
ROUND #1  |  Direction: CLOCKWISE (──►)  |  Win Condition: 500 Pts
Players:  [Ali: 5 cards] ──► [*Erfan (You)*: 4 cards] ──► [Bot_1: 7 cards]
----------------------------------------------------------------
Deck Remaining: 78 cards   |   Pending Draw Penalty: 0
Top Card on Table: [BLUE REVERSE]   |   Active Color: BLUE
----------------------------------------------------------------
Your Hand (Erfan):
  [0] RED 7       [1] BLUE 9       [2] GREEN SKIP       [3] WILD DRAW FOUR (+4)
================================================================
Enter Command > play 1
```

۳. دستورات مجاز بازیکن در حین اجرای راند

`play <card_index>`	بازی کردن کارت عددی یا عملیاتی موجود در اندیس `<card_index>` از لیست پیوندی دست بازیکن
`play <card_index> -c <RED|YELLOW|GREEN|BLUE>`	بازی کردن کارت‌های Wild یا WildDrawFour به همراه تعیین رنگ جدید میز (حساسیت به حروف کوچک و بزرگ وجود ندارد)
`draw`	کشیدن ۱ کارت از Deck در حالت عادی؛ یا دریافت کامل جریمه انباشته‌شده (PenaltyStack) و اتمام نوبت در حالت جریمه
`play_drawn [-c <color>] | pass`	پس از دستور `draw` در حالت عادی، اگر کارت کشیده‌شده قابل بازی باشد، بازیکن می‌تواند با `play_drawn` فوراً آن را بازی کند یا با `pass` نوبت را رد کند
`scores | quit_game`	مشاهده جدول امتیازات راندهای گذشته و مجموع امتیاز فعلی بازیکنان، یا خروج از بازی به منوی اصلی

۳.۷.۴ دستورات دیباگ و چیت‌کدها — ویژه تست و تحویل

برای تسهیل فرآیند تست کیس‌ها و بررسی سریع سناریوهای خاص (مانند خالی شدن `Deck`، زنجیره `Stacking` و پایان راند/بازی) در جلسه تحویل حضوری، پیاده‌سازی دستورات چیت زیر در محیط میز بازی **الزامی** است (اجرای چیت‌کد نوبت بازیکن را جلو نمی‌برد):

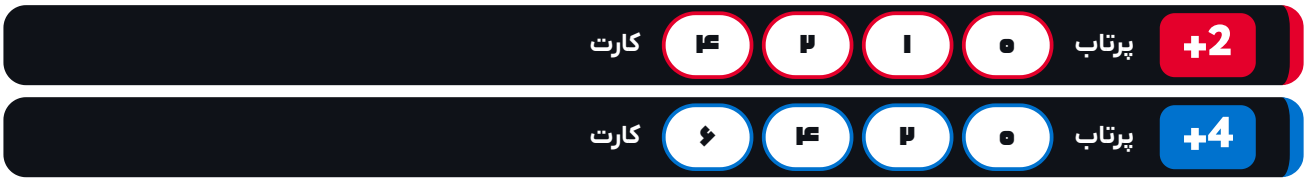
`cheat_give -t <type> -c <color> [-v <0-9>]`	افزودن مستقیم یک کارت دلخواه به انتهای دست بازیکن فعلی؛ مثال: `cheat_give -t WILD_DRAW_FOUR -c NONE` یا `cheat_give -t NUMBER -c RED -v 7`
`cheat_top -t <type> -c <color> [-v <0-9>]`	تغییر مستقیم کارت بالای `DiscardPile` و تنظیم `activeColor` میز بدون تغییر نوبت
`cheat_empty_deck`	حذف کارت‌های داخل صف `Deck` تا زمانی که فقط ۱ کارت در آن باقی بماند (برای تست فوری الگوریتم بازسازی `Deck` از روی `DiscardPile`)
`cheat_win_round`	حذف تمام کارت‌های دست بازیکن فعلی به‌جز ۱ کارت و اعلام پایان فوری راند به نفع بازیکن فعلی (برای تست محاسبه بازگشتی امتیازات)

نمایش خلاصه پایان هر راند

به محض اینکه دست یکی از بازیکنان خالی شد، برنامه باید نام برنده راند، لیست کارت‌های باقی‌مانده در دست هر یک از بازیکنان بازنده، امتیاز محاسبه‌شده (به روش بازگشتی) از هر بازیکن، مجموع امتیاز اضافه‌شده به برنده و جدول امتیازات کل تا آن راند را چاپ کند.

۳.۸ دستگاه پرتاب کارت Draw

در این قابلیت، وقتی بازیکن به‌خاطر کارت‌های `+2` یا `+4` باید جریمه بکشد، می‌تواند به‌جای دریافت مستقیم جریمه از یک «دستگاه شانس» استفاده کند. دستگاه به‌صورت تصادفی تعیین می‌کند که بازیکن چند کارت دریافت خواهد کرد. پیاده‌سازی این بخش الزامی است.

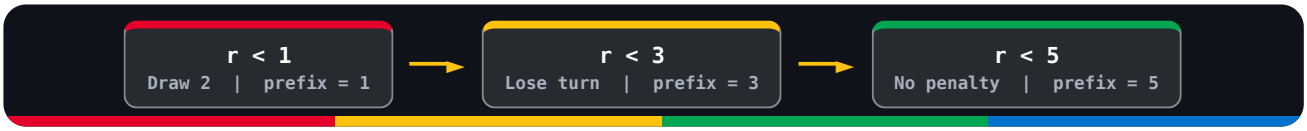


Weighted Random Selection

در انتخاب تصادفی ساده همه‌ی گزینه‌ها شانس یکسان دارند، اما در انتخاب وزن‌دار شانس هر گزینه با وزن آن متناسب است. برای نمونه، سه نتیجه‌ی زیر را در نظر بگیرید:

نتیجه	وزن	احتمال
دریافت ۲ کارت	۱	۲۰٪
از دست دادن نوبت	۲	۴۰٪
بدون جریمه	۲	۴۰٪

مجموع وزن‌ها ۵ است و احتمال هر نتیجه از تقسیم وزن آن بر این مجموع به‌دست می‌آید. برای پیاده‌سازی، جمع‌های تجمعی (prefix sum) وزن‌ها ساخته می‌شود، عدد تصادفی `r` در بازه‌ی `[0, 5)` انتخاب می‌شود و نتیجه اولین خانه‌ای است که جمع تجمعی‌اش از `r` بیشتر باشد. Weighted Random Selection ساختمان داده نیست و یک الگوریتم انتخاب است؛ آرایه‌ی وزن‌ها و جمع‌های تجمعی طبق قانون طلایی باید دست‌ساز باشد:



چه مواردی باید تست شوند؟

۱. وزن صفر باعث انتخاب شدن گزینه نشود.
۲. مجموع وزن‌ها صفر نباشد.
۳. گزینه‌های نامعتبر یا وزن‌های منفی پذیرفته نشوند.
۴. نتیجه‌ی انتخاب‌شده اثر درست داشته باشد.
۵. اجرای تعداد زیادی آزمایش با یک تولیدکننده‌ی تصادفی مناسب، توزیعی نزدیک به احتمال‌های تعریف‌شده نشان بدهد؛ این تست آماری به‌تنهایی اثبات درستی الگوریتم نیست.

۳.۹ اصول کدنویسی تمیز (Clean Code) و نکات تحویل

- **رعایت اصول Clean Code:** نام‌گذاری معنادار کلاس‌ها (`PascalCase`) و متدها/متغیرها (`camelCase`)، کوتاه و تک‌مسئولیتی بودن توابع، عدم تکرار کد (DRY) و تفکیک کامل لایه نمایش از منطق بازی الزامی است.
- **کنترل نسخه در گیت‌هاب (GitHub):** پروژه باید در مخزن گیت‌هاب توسعه یابد و روند پیشرفت کار در قالب کامیت‌های مرحله‌به‌مرحله و معنادار (مثلاً `[feat] implement doubly circular linked list`) ثبت شود. آپلود یکجای پروژه در روز تحویل پذیرفته نمی‌شود.

۳.۱۰ قابلیت‌های امتیازی

پیاده‌سازی موارد زیر اختیاری است و تنها در صورت کسب حداقل ۸۰٪ نمره بخش اصلی پروژه، نمره امتیازی آن‌ها لحاظ خواهد شد:

۳.۱۰.۱ رابط گرافیکی بازی (GUI)

نمایش بصری: دست بازیکن، کارت روی زمین، رنگ فعال، نوبت، جهت گردش و تعداد کارت حریفان.

تعامل کاربر: بازی کارت، کشیدن و اعلام UNO از طریق کلیک و دکمه‌ها.

رویدادها و انیمیشن: اعلان و جلوه‌های بصری برای جریمه، تغییر جهت و تغییر رنگ.

معماری: حفظ الگوی MVC و عدم آسیب به منطق اصلی.

۳.۱۰.۲ مدیریت تاریخچه بازی (Replay & Undo)

سطح ۱ — Replay: بازپخش ترتیبی حرکات و رویدادهای مهم پس از پایان بازی بدون تغییر نتیجه.

سطح ۲ — Undo: لغو آخرین حرکت و بازگرداندن دقیق تمام متغیرهای بازی (پیشنهاد: در حالت تمرینی/آفلاین).

الگوهای معماری: Event Sourcing، Snapshot (هر ۲۰ حرکت) و Command Pattern.

۳.۱۰.۳ بازی آنلاین (Client-Server)

معماری: پیاده‌سازی Client-Server با استفاده از TCP Sockets و مدیریت هم‌زمان کلاینت‌ها با Thread. بازی باید به‌صورت دو یا چندنفره و آنلاین اجرا شود؛ به‌طوری‌که حداقل دو کلاینت به سرور متصل شده و حتماً به‌صورت هم‌زمان با یکدیگر بازی کنند.

پروتکل ارتباطی: مستقل از زبان بر پایه JSON شامل سریال‌سازی و دی‌سریال‌سازی ساختار پیام‌ها و مدیریت خطاهای شبکه.

۳.۱۰.۴ بازیکن هوش مصنوعی (Gemini AI)

ارسال وضعیت: ارسال `GameState` و لیست حرکات مجاز به API جمنای در قالب Prompt ساختاریافته.

اعتبارسنجی سخت‌گیرانه: خروجی مدل نباید مستقیماً اجرا شود؛ `GameEngine` باید پاسخ را اعتبارسنجی کند.

Fallback: در صورت خطای API یا پاسخ نامعتبر، بازی به‌طور خودکار به ربات داخلی سوییچ کند.

۳.۱۰.۵ مودهای جدید بازی

نسخه‌ی پایه فقط قوانین کلاسیک را اجرا می‌کند. در این قابلیت، بازی چند مود دیگر هم پیدا می‌کند که از نسخه‌های مختلف UNO مانند Flip، No Mercy، نسخه‌ی ۱۲۰ کارتی، هری پاتر و لبوبو الهام گرفته‌اند. مود هنگام ساخت بازی و با یک پارامتر جدید در دستور `start_game` انتخاب می‌شود و تا پایان بازی ثابت می‌ماند.

`start_game ... -variant <classic|flip|nomercy|deck120| labubu|magic>`	انتخاب مود بازی. اگر نوشته نشود، `classic` یعنی همان قوانین پایه اجرا می‌شود. پارامتر `-mode` مربوط به شرط پایان بازی است و با این پارامتر فرق دارد.
`-pack <hat|voldemort|howler|all>`	فقط در مود `magic`؛ مشخص می‌کند کدام بسته‌ی کارت‌های جادویی به دسته اضافه شود.

مودهای جدید دو دستور تازه هم لازم دارند که فقط در محیط میز بازی معتبرند:

`play <card_index> -t <player_index> [-c <color>]`	بازی کارت‌هایی که مقصد می‌خواهند (هدف، مودی، تعویض دست‌ها)؛ شماره‌ی بازیکن همان ترتیب خط `Players` میز است.
`react <card_index> | pass`	پاسخ به پرسش برنامه برای بازی کارت واکنشی (سپر، نامرئی یا سنگ رستاخیز) یا رد کردن آن.

هر مود یک مجموعه‌قانون جداست، نه یک `if` در دل موتور

برای اینکه هسته‌ی بازی پر از شرط‌های `variant == ...` نشود، قوانین هر مود در یک کلاس جدا نوشته می‌شود که یک واسط مشترک (مثلاً `RuleSet`) را پیاده می‌کند؛ ساخت دسته، اعتبار حرکت و اثر کارت‌ها از همین کلاس گرفته می‌شود. `Game` هنگام ساخته شدن یکی از این کلاس‌ها را می‌گیرد (الگوی Strategy). این طراحی با اصل جداسازی بخش ۳.۱ هم‌خوان است: مود جدید یک کلاس تازه می‌خواهد، نه تغییر در لایه‌ی Controller.

در ادامه، هر مود به‌همراه نقطه‌ای که از نظر ساختمان داده ارزش آموزشی دارد معرفی شده است. در هر مود، پیاده‌سازی همان ساختارهای دست‌ساز بخش ۳.۲ الزامی است و استفاده از کالکشن‌های آماده همچنان ممنوع است.

مود Flip — دسته‌ی دو رو

`-variant flip`

قوانین

هر کارت یک وجه روشن و یک وجه تاریک دارد و کارت `Flip` همه‌ی کارت‌های میز، `Deck` و دست‌ها را به وجه دیگر می‌برد. وجه روشن `Draw One` و `Wild Draw Two` دارد و وجه تاریک `Draw Five`، `Skip Everyone` و `Wild Draw Color`. امتیاز وجه تاریک بالاتر است: `Skip Everyone` ۳۰ و `Wild Draw Color` ۶۰.

ارزش ساختمان داده‌ای

هر کارت دو وجه دارد و یک پرچم سراسری `activeSide` مشخص می‌کند کدام وجه معتبر است. برگرداندن همه‌ی کارت‌ها فقط تغییر همین پرچم است و **O(1)** هزینه دارد، در حالی که پیمایش و برگرداندن تک‌تک کارت‌ها **O(N)** است. این همان ایده‌ی محاسبه‌ی تنبل (Lazy) است.

مود No Mercy — بدون ترحم

`-variant nomercy`

قوانین

بازیکنی که ۲۵ کارت یا بیشتر در دست داشته باشد حذف می‌شود. جریمه‌ها روی هم انباشته می‌شوند، ولی فقط با کارتی که جریمه‌اش برابر یا بیشتر باشد. کارت‌های `Wild Draw 6`، `Wild Draw 10`، `Skip Everyone`، `Discard All` و `Color Roulette` وارد دسته می‌شوند و بازیکن بدون کارت قابل‌بازی تا رسیدن به یک کارت قابل‌بازی می‌کشد. قانون ۷-۰ هم فعال است: ۷ دست را با یک نفر عوض می‌کند و ۰ دست‌ها را در جهت گردش می‌چرخاند.

ارزش ساختمان داده‌ای

حذف بازیکن یعنی جدا کردن یک گره از `DoublyCircularLinkedList` و انتقال `current`، که با دو اشاره‌گر **O(1)** است. `PenaltyStack` به پشته‌ی یکنواخت تبدیل می‌شود: کارت جدید فقط وقتی `push` می‌شود که مقدارش از `peek()` کمتر نباشد. کارت‌های بازیکن حذف‌شده هم با صف پیوندی دواشاره‌گر در **O(1)** به انتهای `Deck` الحاق می‌شوند.

مود لبوبو — تعویض دست‌ها

`-variant labubu`

قوانین

بازی کلاسیک با یک کارت ویژه‌ی اضافه: `Wild Swap Hands`. بازیکن دست خود را با دست یک نفر دلخواه عوض می‌کند و رنگ جدید بازی را اعلام می‌کند.

ارزش ساختمان داده‌ای

چون دست یک `SinglyLinkedList` با `head`، `tail` و `size` است، تعویض دو دست یعنی جابه‌جا کردن همین سه مقدار؛ **O(1)** و بدون کپی کردن کارت‌ها، در برابر **O(k)** برای روش ساده. برای این کار فیلد `hand` باید داخل `Player` قابل جایگزینی باشد. همین عملیات برای قانون ۷-۰ در مود `nomercy` هم به کار می‌رود.

مود ۱۲۰ کارتی

`-variant deck120`

قوانین

دسته‌ی ۱۲۰ کارتی با سه کارت جدید: `هدف` (هر بازیکن دلخواه ۴ کارت می‌کشد و رنگ تغییر می‌کند)، `سپر` (جریمه‌ی `+2` یا `+4` را خنثی می‌کند) و چهار `کارت سفید` برای قانون خانگی که پیش از توزیع کارت‌ها نوشته و تأیید می‌شود. امتیاز هدف و سپر در پایان راند ۵۰ است.

ارزش ساختمان داده‌ای

هدف‌گیری یعنی پیدا کردن بازیکن مقصد روی لیست حلقوی، **O(P)**. سپر یک کارت واکنشی است و پیش از اعمال جریمه باید دست مقصد برای آن جست‌وجو شود. کارت سفید از ترکیب اثرهای پایه ساخته می‌شود و در یک `DynamicArray` نگه‌داری می‌شود. جزئیات این سه کارت در بخش ۳.۱۰.۶ آمده است.

مود جادویی — هری پاتر

`-variant magic`

قوانین

نسخه‌ی هری پاتر چند مدل دارد: فقط کارت‌های اصلی، بسته‌ی کلاه گروه‌بندی، بسته‌ی ولدمورت، بسته‌ی زوزه‌کش و نامرئی، یا همه‌ی بسته‌ها با هم (سوپرمدل) که مودی و سنگ رستاخیز را هم دارد. بسته با پارامتر `-pack` انتخاب می‌شود.

ارزش ساختمان داده‌ای

دسته دیگر با کد ثابت ساخته نمی‌شود؛ یک جدول مشخصات (نوع، رنگ، تعداد) در `DynamicArray` نگه‌داری می‌شود و هر بسته فقط چند ردیف به آن اضافه می‌کند. سازنده‌ی `Deck` جدول را پیمایش می‌کند، کارت‌ها را می‌سازد و سپس Fisher-Yates را اجرا می‌کند: **O(N)**. افزودن یک کارت جدید یعنی افزودن یک ردیف، نه تغییر در موتور.

۳.۱۰.۶ کارت‌های جدید

کارت‌های زیر از نسخه‌های ۱۲۰ کارتی، هری پاتر، لبوبو، Flip و No Mercy گرفته شده‌اند. هر کدام یک عملیات تازه روی ساختمان داده‌های پایه می‌خواهد و همین ارزش آموزشی‌شان را می‌سازد. زیر نام هر کارت، مودهایی که آن را در دسته می‌گذارند نوشته شده است.

کارت و مود	اثر در بازی	عملیات روی ساختمان داده
هدف · **مودی** `deck120` `magic`	بازیکن هر نفر دلخواه را انتخاب می‌کند تا ۴ کارت (هدف) یا ۲ کارت (مودی) بکشد و رنگ بعدی را هم صاحب کارت تعیین می‌کند.	پیدا کردن مقصد روی لیست حلقوی با پیمایش، **O(P)**. جریمه به‌جای «نفر بعدی» به مقصد انتخاب‌شده داده می‌شود.
سپر · **نامرئی** `deck120` `magic`	کارت واکنشی: سپر جریمه‌ی `+2` یا `+4` را خنثی می‌کند و نامرئی اثر کارت‌هایی مثل زوزه‌کش را. روی `Skip` ، `Reverse` و هدف اثری ندارند.	پیش از اعمال جریمه، دست مقصد برای کارت واکنشی پیمایش می‌شود، **O(k)**. با استفاده از سپر، `PenaltyStack` به‌صورت بازگشتی خالی می‌شود، **O(m)**.
تعویض دست‌ها `labubu` `nomercy`	`Wild Swap Hands` در مود لبوبو و عدد ۷ در `nomercy` : دست بازیکن با دست یک نفر دلخواه عوض می‌شود. عدد ۰ دست همه را یک جایگاه در جهت گردش می‌چرخاند.	تعویض `head` ، `tail` و `size` دو لیست، **O(1)**. چرخش دست‌ها با P−1 تعویض پشت‌سرهم روی حلقه‌ی بازیکنان، **O(P)**.
حذف دسته‌ای · **ولدمورت** `nomercy` `magic`	`Discard All` همه‌ی کارت‌های هم‌رنگ با کارت بازی‌شده را از دست بازیکن بیرون می‌ریزد و ولدمورت همه‌ی کارت‌های هم‌نوع با کارتی را که بازیکن مشخص می‌کند.	یک تابع `removeIf(condition)` روی `SinglyLinkedList` با یک پیمایش و بازبستن اشاره‌گرها، **O(k)**؛ کارت‌های حذف‌شده روی `DiscardPile` `push` می‌شوند.
کشیدن تا رسیدن به رنگ `flip` `nomercy`	`Wild Draw Color` (وجه تاریک `flip`)، `Color Roulette` (`nomercy`) و کلاه گروه‌بندی (`magic`): مقصد تا رسیدن به کارتی از رنگ انتخاب‌شده از `Deck` می‌کشد.	حلقه‌ی `dequeue` روی `Deck` ؛ بدترین حالت **O(N)**. اگر وسط کار `Deck` خالی شود، بازسازی بخش ۳.۴ همان‌جا اجرا می‌شود.
نوبت اضافه `flip` `nomercy`	`Skip Everyone` : همه‌ی بازیکنان رد می‌شوند و نوبت به صاحب کارت برمی‌گردد.	`advanceTurn(P)` روی حلقه‌ی P نفره دور کامل می‌زند؛ با گرفتن باقیمانده‌ی `steps % size` هزینه **O(1)** می‌شود، نه **O(P)**.
زوزه‌کش `magic`	مقصد باید همه‌ی کارت‌های دستش را برای همه نشان دهد، مگر اینکه با کارت نامرئی اثر را خنثی کند.	یک پیمایش فقط‌خواندنی روی دست مقصد، **O(k)**. چاپ دست وظیفه‌ی View است و نباید دست را تغییر دهد.
سنگ رستاخیز `magic`	کارت واکنشی؛ وقتی بازیکنی آخرین کارتش را بازی می‌کند، صاحب سنگ می‌تواند او را به بازی برگرداند: آخرین کارت او از `DiscardPile` به دستش برمی‌گردد و ۲ کارت جریمه هم می‌گیرد.	`pop()` از پشته و `addLast` به دست، هر دو **O(1)**. بررسی این کارت پیش از محاسبه‌ی امتیاز راند انجام می‌شود.
کارت سفید `deck120`	قانون خانگی که پیش از شروع بازی ثبت و توسط همه تأیید می‌شود؛ هر کارت سفید یک قانون دارد.	هر قانون ترکیبی از اثرهای پایه (کشیدن، پرش، تعویض و...) است که در `DynamicArray` نگه‌داری و به ترتیب اجرا می‌شود، **O(e)** برای e اثر.

اعمال جریمه یا اثر ← مقصد کارت واکنشی دارد؟ ← پرسش از بازیکن ← خنثی شدن یا اجرا